DHBW Stuttgart

Karsten Keßler

Datenbanksysteme
10. Teil – Transaktionen

Kurs Informatik 24ITA

Thema	Inhalte (Auszug)
1 Einführung	Einführung, Begriffe, Eigenschaften Funktionen, Datenintegration
2 Datenbankmodelle	Architektur und SPARC, Verschiedene Datenbankmodelle, Entwicklung
3 ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
4 Relationales Modell	Relationales Datenbankmodell im Detail
5 Relationale Algebra	Operatoren, Anwendung, Nützlichkeit
6 Normalformen	Normalisierung von Datenbanken, funktionale Abhängigkeiten, Synthesealgorithmus
7 Vektordatenbanken	Einstieg in Vektordatenbanken
8 SQL DDL	Datendefinition mit SQL
9 SQL DML	Datenmanipulation mit SQL
10 Transaktionen	ACID, Grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
11 Integrität	Integrität im relationalen Modell wiederholt, Umsetzung (in SQL)
12 Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
13 Indizes	Interne Strukturen, Nützlichkeit für Anfragen
14 Optimierung	Optimierung von Datenbanken
15 SQL Server	Administration des SQL Server
16 Handout	Handout: Datenbanksysteme – Zusammenfassung

ACID Kriterien für Transaktionen

Atomicity

Eine Transaktion ist die kleinste, nicht mehr weiter zerlegbare Einheit. Sie wird vollständig oder gar nicht ausgeführt.
Bsp: Ein Bankkunde überweist Geld von Konto A auf Konto B. Falls ein Fehler auftritt (z. B. ein Serverabsturz), wird sichergestellt, dass entweder beide Buchungen (Lastschrift von A, Gutschrift auf B) durchgeführt oder keine der beiden ausgeführt wird.

Consistency

Nach Durchführung der Transaktion ist die Datenbank wieder in einem konsistenten Zustand, ansonsten wird sie zurückgesetzt.
Bsp: Eine Datenbank enthält eine Einschränkung, dass der Kontostand nicht negativ sein darf. Falls eine Transaktion den Kontostand ins Negative bringen würde, wird sie abgebrochen.

Isolation

Nebenläufige Transaktionen beeinflussen sich nicht gegenseitig.
Jede wird „logisch“ ausgeführt als gäbe es keine andere.
Bsp: Zwei Kunden tätigen gleichzeitig Transaktionen auf dasselbe Konto. Isolation stellt sicher, dass sie sich gegenseitig nicht beeinflussen und die Endergebnisse korrekt bleiben.

Durability

Dauerhaftigkeit (Persistenz) durchgeführter Aktionen muss gewährleistet sein. Alle späteren Fehler betreffen die hier durchgeführten Änderungen nicht.
Bsp: Nach Abschluss einer Überweisung bleibt die Änderung auch bei einem Stromausfall oder Systemabsturz gespeichert, da sie auf die Festplatte oder ein anderes sicheres Speichermedium geschrieben wurde.

DBS übernimmt die Transaktionsverwaltung und gewährleistet ACID.

ACID sorgt dafür, dass CRUD-Operationen sicher und zuverlässig ausgeführt werden. CRUD allein beschreibt nur grundlegende Operationen, während ACID die korrekte und konsistente Ausführung sicherstellt. In relationalen Datenbanken werden CRUD-Operationen meist in Transaktionen verpackt, um ACID zu gewährleisten.

CRUD - Operationen

- **C**reate — Erstelle einen Datensatz / Zeile in einer Tabelle
- **R**ead — Lese einen Datensatz / Zeile in der Tabelle (oder mehrere)
- **U**pdate — Verändere einen Datensatz / Zeile in der Tabelle (oder mehrere)
- **D**elete — Lösche einen Datensatz / Zeile in der Tabelle (oder mehrere, oder die Tabelle, oder die DB)

DB-CRUD	HTTP-Action
Create (Insert)	POST
Read (Select)	GET
Update	PUT
Delete	DELETE

So sieht eine Anwendung von CRUD in Web-APIs aus:

```
# Route zum Erstellen eines neuen Benutzers
@app.route('/users', methods=['POST'])
def create_user():
    data = request.get_json()
    new_user = User(name=data['name'])

    db.session.add(new_user)
    db.session.commit()
    .....
```

Umsetzen von ACID Kriterien in MySQL

ACID-Kriterium	SQL	Beispiel
Atomicity	Die Transaktion wird entweder vollständig ausgeführt oder gar nicht. Falls ein Fehler auftritt, wird alles zurückgesetzt.	START TRANSACTION; INSERT INTO users ...; COMMIT; oder ROLLBACK; falls ein Fehler passiert.
Consistency	Die Datenbank bleibt immer in einem gültigen Zustand (keine doppelten IDs, keine ungültigen Werte).	TRANSAKTIONEN, aber auch PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, TRIGGER, CHECK-Constraints stellen sicher, dass das INSERT keine Inkonsistenzen erzeugt.
Isolation	Gleichzeitige INSERTs beeinflussen sich nicht gegenseitig, sodass keine doppelten oder fehlerhaften Einträge entstehen.	SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; verhindert, dass zwei parallele Transaktionen denselben Benutzer einfügen.
Durability	Nach COMMIT bleibt der neue Benutzer auch nach einem Absturz dauerhaft gespeichert.	MySQL-spezifisch: innodb_flush_log_at_trx_commit = 1; schreibt die Transaktion sofort auf die Festplatte.

```
try:
    conn.start_transaction()
    conn.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
    conn.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 2")
    conn.commit()
except:
    conn.rollback()
```

```
ALTER TABLE accounts ADD CONSTRAINT positive_balance CHECK (balance >= 0);
```



```
SHOW ENGINES;
SHOW VARIABLES LIKE 'innodb_flush_log_at_trx_commit';
```

Engine	Support	Comment	Transactions	XA	Savepoints
MEMORY	YES	Hash based, stored in memory,...	NO	NO	NO
MRG_MYISAM	YES	Collection of identical MyISAM t...	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
InnoDB	DEFAULT	Supports transactions, row-leve...	YES	YES	YES
ndbinfo	NO	MySQL Cluster system informat...	NULL	NULL	NULL
BLACKHOLE	YES	/dev/null storage engine (anyth...	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
ndbcluster	NO	Clustered, fault-tolerant tables	NULL	NULL	NULL

Variable_name	Value
innodb_flush_log_at_trx_commit	1

InnoDB ist die Standard-Storage-Engine in MySQL, die ACID-Transaktionen unterstützt und für hohe Zuverlässigkeit und Performance optimiert ist.

Beispiel:Geldüberweisung mit Transaktionslogik (z. B. im Onlinebanking oder Geldautomaten)

- Kunde überweist Firma 100 EUR
- Firma wird 100 EUR gutgeschrieben
- System stürzt ab

Transaktion: logische Verarbeitungseinheit, die vollständig ausgeführt werden muss, um Korrektheit sicherzustellen*; dafür wird benötigt: Synchronisation (mehrere Transaktionen) und Recovery

Fehlerquellen für Transaktionen:

- Rechnerfehler (System stürzt ab), HW, SW, Netzwerk
- Transaktionsfehler / Systemfehler: Überlauf einer Zahl, Division durch Null, Programmierfehler, Abbruch durch Benutzer
- Ausnahmebedingung: z. B. Wert wird nicht gefunden, oder Saldo zu klein (=> guter Entwurf sollte das berücksichtigen)
- Nebenläufigkeit: Abbruch durch System / Synchronisation (siehe später)
- Plattenfehler: Datenverlust auf der Platte
- Physikalische Fehler / Katastrophen: Brand, Lüftung, Sabotage, ..

SQL und Transaktionsunterstützung

- **Zugriffsmodus** (access mode)
  - READ ONLY (SET TRANSACTION READ ONLY)
  - READ WRITE
- **Isolationsstufen**
  - Siehe Tabelle
- **SQL**

  set transaction [{read only | read write},]

  [isolation level { ..},]
  [diagnostics size …] (Anzahl Diagnoseinträge)

Isolationsstufe	Dirty Read	Non-Repeatable Read	Lost Update	Phantom Read
READ UNCOMMITED	möglich	möglich	möglich	möglich
READ COMMITED	---	möglich	möglich	möglich
REPEATABLE READ	---	---	---	möglich
SERIALIZABLE	---	---	---	---

- **SQL: Integrität** (wann wird eine aufschiebbare Einschränung überprüft)
  set constraints [{immediate | deferred}]

Datenkonsistenz nimmt zu
Je höher die Isolationsebene, desto weniger Anomalien treten auf
READ UNCOMMITTED = viele Anomalien möglich → schlechte Konsistenz
SERIALIZABLE = höchste Isolation → beste Datenkonsistenz

Performance nimmt ab
Je höher die Isolation, desto mehr Sperren / Prüfungen / Wiederholungen
SERIALIZABLE blockiert ggf. mehr Transaktionen oder verursacht mehr Wartezeit
READ UNCOMMITTED erlaubt maximale Parallelität (aber auf Kosten der Korrektheit)

SERIALIZABLE: DBMS führt jede Transaktion aus als fände jede Transaktion seriell (hintereinander) statt. Realsierung z.B. Durch Locking-Mechanismen oder Snapshots.

REPEATABLE READ: DBMS muss einzelne zu lesende bzw. zu ändernde Datensätze vor Zugriffen durch andere Transaktionen bis zu deren Ende schützen. Üblicherweise durch Read und Write Locks.
In MySQL (InnoDB) wird hier MVCC/Snapshot Isolation genutzt. Transaktionen sehen konsistente, aber ggf. veraltete Sichten.

READ COMMITED: Auch hier muss das DBMS Write-Locks bis zum Ende der Transaktion garantieren. Read Locks werden aber sofort nach der Ausführung der SELECT-Anweisung aufgelöst. Deswegen kann es hier passieren, dass zwei SELECT-Anweisungen innerhalb einer Transaktion zu verschiedenen Ergebnissen kommen.

READ UNCOMMITED: Auch kann es vorkommen, dass eine Transaktion A Änderungen einer Transaktion 2 sieht, selbst wenn 2 noch gar nicht committed ist.

Datenkonsistenz nimmt zu:
→ Je höher die Isolationsebene (von *READ UNCOMMITTED* bis *SERIALIZABLE*), desto besser ist die Transaktion gegen Anomalien abgesichert.
Performance nimmt ab:
→ Je höher die Isolationsebene, desto mehr Sperren und Einschränkungen entstehen – das reduziert die Parallelität und damit die Performance.

Fehler bzw. Anomalien

Dirty Read: Transaktion 1 sieht veränderte Daten, die sich durch eine zweite, noch gar nicht abgeschlossene Transaktion 2 ergeben.

Non-Repeatable Read: Mehrere gleichwerte SQL-Abfragen innerhalb einer Transaktion kommen zu unterschiedlichen Ergebnissen.

Lost Update: Zwei Transaktionen verändern unabhängig voneinander denselben Datensatz. Letztendlich wird aber nur eine Änderung dauerhaft abgespeichert.

Phantom Read: Ist eine Variante von Non-Repeatable Read, allerdings ergibt sich der Fehler durch zwei unterschiedliche SQL-Befehle.

Lost Update

Zwei parallele Banktransaktionen, beide arbeiten mit der gleichen Ausgangsbasis

A1: Abbuchung	A2: Zinsgutschrift
ka = Read(A,“kontostand“)	
ka = ka - 100	
	ka = Read(A,“kontostand“)
...	ka = ka * 1.03
	WRITE (A, „kontostand“, ka)
WRITE (A, „kontostand“, ka)	
...	

Zeitachse

Transaktion A2 geht verloren !

A1 (Abbuchung) liest den Kontostand ka = Read(A, "kontostand") → z. B. 1000
A1 zieht 100 ab → ka = ka - 100 → 900
A2 (Zinsgutschrift) liest ebenfalls den gleichen ursprünglichen Kontostand → ka = 1000
A2 rechnet 3 % Zinsen drauf → ka = 1030
A2 schreibt 1030 zurück
A1 schreibt 900 zurück → die Änderung von A2 geht verloren

Was kann man tun?
Transaktionen mit Sperren (z. B. SELECT FOR UPDATE)
Verwendung eines höheren Isolation Levels (z. B. Repeatable Read oder Serializable)

Dirty Read

Zwei parallele Banktransaktionen,
Eine Transaktion liest Daten, die von einer anderen Transaktion geändert, aber noch nicht bestätigt wurden.

A1: Umbuchung	A2: Zinsgutschrift (schreibt mit falschem Wert)
ka = Read(A,“kontostand“)	
ka = ka - 100	
WRITE (A, „kontostand“, ka)	
...	
	ka = Read(A,“kontostand“)
	ka = ka * 1.03
	WRITE (A, „kontostand“, ka)
kb = Read(B,“kontostand“)	
Abort (abgebrochen, nicht comitted)	

Zeitachse

A1 muss rückgesetzt werden. Zins wurde aber auf „ungültigen“ Daten berechnet!

Was kann man tun?
Mindestens das Isolation Level
READ COMMITTED verwendet (statt READ UNCOMMITTED).

Non-Repeatable Read

Eine Transaktion liest dieselbe Zeile mehrmals, aber bekommt unterschiedliche Werte, weil zwischenzeitlich eine andere Transaktion den Wert verändert hat.

Die abgefragten Summen sind unterschiedlich!

Zwei parallele Banktransaktionen

A1: Kontoeröffnung	A2: Abfrage
ka = Read(A,“kontostand“)	
...	WRITE (A, ...)
ka = Read(A,“kontostand“)	

Zeitachse

A1 liest den Kontostand ka = Read(A, "kontostand") → z. B. 1000
A2 ändert den Kontostand → WRITE(A, ...) → z. B. 1100
A1 liest erneut → ka = Read(A, "kontostand") → jetzt 1100

Was kann man tun?
REPEATABLE READ oder SERIALIZABLE als Isolation Level
Sperren auf Lesedaten (z. B. SELECT ... FOR SHARE)

Phantome

Eine Transaktion sieht beim wiederholten Ausführen einer Abfrage zusätzliche oder fehlende Zeilen, die beim ersten Mal nicht da waren – weil sie neu eingefügt oder gelöscht wurden von einer anderen Transaktion.
Phantom Reads betreffen Unterschiede in Ergebnismengen (z. B. neue Zeilen durch INSERT) – im Gegensatz zu Non-Repeatable Read, das Werte einzelner Zeilen betrifft.

Konto C hätte berücksichtigt werden müssen, war aber vorher gar nicht vorhanden!

Zwei parallele Banktransaktionen

A1: Kontoeröffnung	A2: Abfrage
SELECT sum(Kontostand) FROM Konten	
...	INSERT INTO Konten VALUES (C, 1000, ...)
SELECT sum(Kontostand) FROM Konten	

Zeitachse

Was kann man tun?
SERIALIZABLE Isolation Level (höchste Stufe)
Oder mit Range Locks (wenn DBMS sie unterstützt)

Ausführungspläne und Konflikte

Ausführungsplan: Transaktionen (T1, .., Tn): jede Transaktion enthält mehrere Operationen (Transaktionsoperationen) wie Lesen, Schreiben, …. Diese müssen in der gleichen Reihenfolge im Ausführungsplan vorkommen, wie sie in der Transaktion beschrieben sind.

Wann kann es zu einem **Konflikt** kommen?

1. Unterschiedliche Transaktionen
2. Gleiches Objekt (Tupel, Datensatz, Zeile, Wert)
3. Eine davon ist schreibend (also nicht „READ“)

Op T1	Op T2	Problem?	Grund
READ(X)	READ(X)	nein	Nur lesend
READ(X)	WRITE(X)	Ja	Daten können verändert werden
WRITE(X)	READ(X)	ja	Leser könnte inkonsistente Daten bekommen
WRITE(X)	WRITE(X)	Ja	Letzter überschreit (Lost Update!)

Mögliche Lösungen

- Alle Transaktionen hintereinander ausführen
  - **Serieller Ausführungsplan**
  - Per Definition korrekt (unabhängig davon, welche Reihenfolge die Transaktionen haben)
  - Ineffizient bei vielen parallelen Transaktionen
- Alternative: **Serialisierbar (!= Seriell!)**
  - Genau dann wenn zu einem (möglichen, es gibt mehrere) seriellen Ausführungsplan *äquivalent* ist

Das Ziel besteht darin, Transaktionen oder DB-Operationen parallel auszuführen und dennoch zu garantieren, dass das Ergebnis mit einer seriellen Ausführung übereinstimmt.

→ das ist eine der zentralen Aufgaben eines Datenbankmanagementsystems (DBMS) – insbesondere im Bereich der Transaktionsverwaltung

Schritt	T_1	T_2
1.	**BOT**	
2.	Read(A)	
3.		**BOT**
4.		Read(C)
5.	Write(A)	
6.		Write(C)
7.	Read(B)	
8.	Write(B)	
9.	**Commit**	
10.		Read(A)
11.		Write(A)
12.		**Commit**

Schritt	T_1	T_2
1.	**BOT**	
2.	Read(A)	
3.	Write(A)	
4.	Read(B)	
5.	Write(B)	
6.	**Commit**	
7.		**BOT**
8.		Read(C)
9.		Write(C)
10.		Read(A)
11.		Write(A)
12.		**Commit**

Schritt	T_1	T_2
1.	**BOT**	
2.	Read(A)	
3.	Write(A)	
4.		**BOT**
5.		Read(A)
6.		Write(A)
7.		Read(B)
8.		Write(B)
9.		**Commit**
10.	Read(B)	
11.	Write(B)	
12.	**Commit**	

Die Operationen werden so gruppiert, dass T_1 zuerst vollständig abgearbeitet wird. Danach folgt T_2 → Das ist eine serielle Reihenfolge: T_1 → T_2

Verzahnung, die nicht serialisierbar ist: Hier sind die Operationen von T_1 und T_2 ineinander verschränkt. Es gibt zyklische Konflikte:
T_1 liest A, T_2 schreibt A → T_1 vor T_2
T_2 liest B, T_1 schreibt B → T_2 vor T_1
Das ist ein Zyklus im Konfliktgraph → nicht serialisierbar!

Serialisierbarkeit – Sicherstellung mit verschiedenen Methoden

- **Protokolle:** Datenbanken nutzen Regeln (Protokolle), um die Ordnung und Sicherheit beim Zugriff auf Daten zu gewährleisten.
  - **Sperren** auf Objekte: Eine Transaktion darf ein Objekt (z. B. Zeile) exklusiv sperren, um Konflikte zu vermeiden.
  - Nutzung von **Zeitstempeln**, so kann eine DB entscheiden wer zuerst war und wie Konflikte zu behandeln sind.
- **Rücksetzbar** (recoverable): Es können Transaktionen abgebrochen werden, ohne dass andere, die mit commit abgeschlossen wurden, betroffen sind
  - Erst dann commit, wenn alle, von denen sie gelesen hat, beendet sind.
- **Kaskadierendes Rücksetzen:** Das Rücksetzen einer Transaktion löst weitere Rücksetzungen aus.
  (T1 schreibt → T2 liest → T1 wird zurückgesetzt → T2 muss ebenfalls zurückgesetzt werden)
  - Änderungen werden erst nach dem commit freigegeben.
- **Strikt:** Es dürfen auch veränderte Daten einer noch laufenden Transaktion nicht überschrieben / gelesen werden.
  Strenge Regel: Daten, die von einer Transaktion verändert wurden, dürfen erst dann gelesen oder überschrieben werden, wenn sie committed ist.

DHBW Stuttgart

Locking-Verfahren

Einige Abkürzungen

Abkürzung	Bedeutung	
2PL	Two-Phase Locking	Klassisches Sperrprotokoll mit Sperr- und Freigabephase zur Gewährleistung von Serialisierbarkeit
BOT	Begin of Transaction	Startzeitpunkt einer Transaktion
EOT	End of Transaction	Zeitpunkt des COMMIT oder ROLLBACK
IS-Lock	Intend Shared Lock	Absicht, später einen Shared Lock auf ein Objekt zu setzen (bei Hierarchie-Sperren)
IX-Lock	Intend Exclusive Lock	Absicht, später einen Exclusive Lock zu setzen (bei Hierarchie-Sperren)
MVCC	Multiversion Concurrency Control	Verfahren, bei dem jede Transaktion eine eigene Sicht auf die Daten hat (z. B. in PostgreSQL)
OCC	Optimistic Concurrency Control	Sperrt nichts aktiv, sondern prüft am Ende, ob Konflikte vorlagen (z. B. bei Web-APIs)
S-Lock	Shared Lock	Lesesperre: mehrere Transaktionen dürfen gleichzeitig lesen
SS2PL	Strict Two-Phase Locking	Sperren werden erst nach Commit/Abort freigegeben → verhindert kaskadierendes Zurücksetzen
TX	Transaction	Allgemeine Bezeichnung für eine Transaktion
X-Lock	Exclusive Lock (Write Lock)	Schreibsperre: nur eine Transaktion darf schreiben (und lesen)

DHBW Stuttgart

Wie kann eine DBMS die verschiedenen Isolation Level garantieren? Verschiedene DBMS haben verschiedene Locking-Mechanismen implementiert, z.B.

- **Beispiel Twp-Phase-Locking (dt. Zwei-Phasen-Sperrprotokoll, 2PL):**

Grundidee: bei jeder Transaktion wird zwischen zwei Phasen unterschieden. Im ersten Abschnitt werden alle für die Transaktion erforderlichen Locks angefordert. Im zweiten Abschnitt werden diese Locks wieder freigegeben. Sobald eine Transaktion den ersten Lock freigibt, kann sie keine neuen Locks mehr anfordern. Dadurch kann eine Transaktion nicht in beliebiger Reihenfolge Locks erwerben und anfordern.

1. Wachstumsphase: Alle nötigen Sperren (Locks) werden gesetzt.
2. Schrumpfphase: Sobald die erste Sperre freigegeben wird, dürfen keine neuen Sperren mehr gesetzt werden.

→ Dadurch wird verhindert, dass Transaktionen in beliebiger Reihenfolge auf Daten zugreifen und sich gegenseitig stören. Das sichert Serialisierbarkeit.

Wichtig:
- Sperren gelten mindestens bis Transaktionsende.
- Bei **Strict 2PL** bleiben alle Schreibsperren bis zum COMMIT/ROLLBACK aktiv.
- Verwendet Shared Locks (S) und Exclusive Locks (X).

```
LOCK(x);        → Sperre setzen (Phase 1)
LOCK(y);
UNLOCK(y);      → erste Freigabe → Phase 2 beginnt
LOCK(z);        → Verboten! Regel verletzt
```

Locking

Row Locking: einzelne Datenstätze sperren (z.B. UPDATE table SET data = 2 where id = 1)

Range Locking: Falls klar definierter Bereich betroffen ist, verwenden manche DBMS ein Range Locking (z.B. UPDATE …. WHERE id BETWEEN 100 AND 150)

Page Locking: ganze Speicherseiten werden blockiert, in denen Datensätze gespeichert sind.

Table Locking: Wenn klare Zuordnung nicht möglich ist, muss die ganze Tabelle gesperrt werden (z.B. UPDATE table SET price = price x 1.13)

Grundsätzlich wird zwischen zwei Arten unterschieden:

Write Lock (Exclusive Lock, X): Sperrung von Datensätze, die eine Transaktion verändern möchte. Write Locks blockieren andere Transaktionen, die dieselben Datensätze lesen oder schreiben möchten)

Read Locks (Shared Locks, S): Datensätze werden nur so weit gesperrt, dass die eigene Transaktion sie sicher lesen kann. Andere Transaktionen, die auch nur Lesezugriffe durchführen, können ungehindert arbeiten. Transaktionen, die den gleichen Datensätze verändern möchten, werden aber blockiert, bis der Read Lock aufgelöst wird, die betreffende Transaktion also beendet wird.

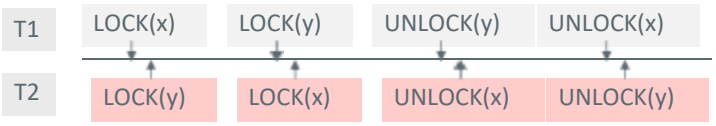
Sperrtyp	Beschreibung	Beispiel
Row Locking	Sperrt einzelne Zeilen	UPDATE table SET data = 2 WHERE id = 1
Range Locking	Sperrt einen Bereich von Zeilen (z. B. bei BETWEEN, >/<) – kommt oft bei SERIALIZABLE vor	WHERE id BETWEEN 100 AND 150
Page Locking	Sperrt ganze Speicherseiten im Speicher (typisch für DBMS wie SQL Server oder Oracle)	Wird intern verwendet
Table Locking	Sperrt ganze Tabelle, wenn keine feinere Zuordnung möglich oder erwünscht ist	UPDATE table SET price = price * 1.13

Verklemmungen (Deadlock) in Synchronisationsverfahren

Zeit	Transaktion T1	Transaktion T2
t_1	LOCK(X)	
t_2		LOCK(Y)
t_3	LOCK(Y) (blockiert)	LOCK(X) (blockiert)

T1 hält Lock auf X und will Y
T2 hält Lock auf Y und will X
→ Zyklus: keiner kommt mehr weiter
→ Deadlock

Beispiel :



T1 hält X und will Y
T2 hält Y und will X
→ Klassischer Deadlock

Deadlocks sind *inhärentes* Problem aller sperrbasierten Synchronisationsverfahren

Beispiele für Umgang mit Verklemmungen

A) Timeout: Abbruch wenn T länger als t_{max} auf eine Sperre wartet

B) Wartegraphen: Kanten: T1 wartet auf T2 ... Ein Zyklus im Graphen entspricht einer Verklemmung - Löschen einer Transaktion auf dem Graphen!

C) Preclaiming: Alle Sperren zu Beginn anfordern!
Problem: Obermengen/Große Sperrbereiche

D) Wait Depth Limit: z.B. Warten nur auf laufende Transaktionen (Wartetiefe: Wie viele „Stufen“)

E) Zeitmarken: Wait/Die- und Wound/Wait-Verfahren

Entstehen Konfikte?

Konfliktverhalten zwischen zwei Transaktionen, die unterschiedliche Arten von Locks auf dieselben Daten erfordern:

Lock, den die nächste Transaktion T2 erfordert: ↓

Existierender Lock aus T1 →

	Shared Lock (S)	Exklusive Lock (X)
Kein vorhandener Lock	OK	OK
Shared Lock (S)	OK	---
Exclusive Lock (X)	---	---

Wait-Die und Wound-Wait

In MySQL ist standardmäßig ein **Wait-Die**-ähnliches Verhalten implementiert:
Die wartende Transaktion wird nach einem Timeout abgebrochen.

Strategie	Alt (T1)	Jung (T2)
Wait-Die	Darf warten	Wird abgebrochen
Wound-Wait	Bricht andere ab („wound“)	Muss warten

Eine Transaktion T_1 versucht ein Objekt zu sperren, dass aber schon von einer anderen Transaktion T_2 gesperrt ist.

- **Wait-Die (**„Wait if older – Die if younger“, Warten-Sterben, Fokus liegt auf der anfragenden (neuen) Transaktion):
  Ältere Transaktion darf auf eine Sperre warten, jüngere Transaktion wird bei Konflikt abgebrochen (Rollback)
- **Wound-Wait (**„Wound if older – Wait if younger“ Verwunden-Warten, Fokus liegt auf der haltenden (blockierenden Transakt,**)**:
  Ältere Transaktion kann eine jüngere abbbrechen (“verwunden”), jüngere muss bei älteren warten.
- Beide Fälle sind verklemmungsfrei
  Wait-Die: Entscheidung liegt bei anfragenden Transaktion
  Wound-Wait: Entscheidung liegt bei haltenden Transaktion

Wait-Die: Ältere Transaktion darf warten, jüngere wird zurückgesetzt
(nur Simulation):

```
-- Session 1 (ältere Transaktion mit früherem Zeitstempel)
START TRANSACTION;  -- Transaktion beginnen
SELECT * FROM data WHERE id = 1 FOR UPDATE;
-- Exklusive Sperre auf Zeile mit id=1 anfordern und erhalten
SELECT SLEEP(60);  -- 60 Sekunden warten, simuliert längere
-- Verarbeitung während die Sperre gehalten wird

-- Session 2 (jüngere Transaktion mit späterem Zeitstempel)
START TRANSACTION;  -- Neue Transaktion beginnen
SELECT * FROM data WHERE id = 1 FOR UPDATE;  -- Versucht,
-- dieselbe Zeile zu sperren, wird blockiert
-- gemäß Wait-Die-Regel: T2 ist jünger und muss sich beenden ("sterben")
ROLLBACK;
-- T2 bricht ab und muss später neu versuchen

-- T2 ist jünger → muss zurückgesetzt werden laut
-- Wait-Die-Protokoll
-- In einem echten System würde dies automatisch
-- geschehen, hier manuell simuliert
```


Wound-Wait: Ältere Transaktion 'verwundet' jüngere, jüngere darf nur warten
(nur Simulation):

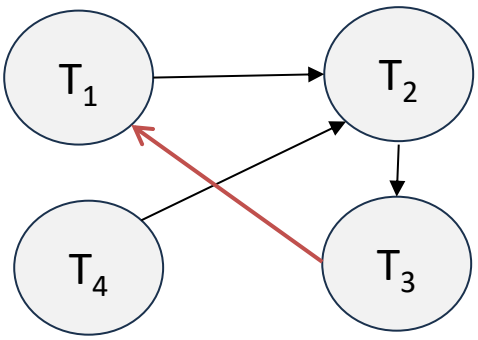
```
-- Session 2 (ältere Transaktion mit früherem Zeitstempel)
START TRANSACTION;  -- Transaktion beginnen
SELECT * FROM data WHERE id = 1 FOR UPDATE;  -- Exklusive Sperre
--- auf Zeile mit id=1 anfordern und erhalten
SELECT SLEEP(60);  -- 60 Sekunden warten, simuliert längere Verarbeitung

-- Session 1 (jüngere Transaktion mit späterem Zeitstempel)
START TRANSACTION;  -- Neue Transaktion beginnen
SELECT * FROM data WHERE id = 1 FOR UPDATE;  -- Versucht, dieselbe Zeile
-- zu sperren, wird blockiert
-- gemäß Wound-Wait-Regel: T1 ist jünger und würde von T2 "verwundet" werden
ROLLBACK;  -- T1 wird abgebrochen, da ältere T2 Vorrang hat

-- T1 ist jünger → darf nicht blockieren, wird abgebrochen
-- In einem echten Wound-Wait-System würde die ältere T2 die
-- jüngere T1 aktiv abbrechen
-- (hier manuell simuliert für Demonstrationszwecke)
```

Erkennung von Deadlocks (Abhängigkeiten am Wartegraph)

	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9
T_1	S(A)	S(D)		S(B)					
T_2			X(B)				X(C)		
T_3					S(D)	S(C)			X(A)
T_4								X(B)	



1. T1 bekommt Shared Lock auf A
2. T1 bekommt Shared Lock auf D
3. T2 bekommt Exlusiv Lock auf B
4. T1 will Shared auf B –> T2 hält B, T1 muss warten (T1→T2)
5. T3 bekommt D
6. T3 bekommt C
7. T2 will C, ist aber von T3 gesperrt, T2 muss warten (T2→T3)
8. T4 will B, ist aber bei T2, T4 muss warten (T4 →T2, nicht im Zyklus)
9. T3 will A, ist aber von T1 gesperrt, T3 muss warten (T3→T1)

→ Deadlock: T1→ T2→T3→T1

Moderne Sperrverfahren (MVCC)

Statt Datensätze zu überschreiben, richtet das DBMS eine neue Version ein. Es kann also mehrere Versionen eines Datensatzes geben (sogar für jede Transaktion eine).

- **Multiversionsnebenläufigkeitscontrolle (MVCC)**
- Mehr Speicherplatz notwendig, außer man hält bereits verschiedene Werte vor (Stichwort temporale DB)
- Erlaubt es, ältere Versionen zu lesen (was sonst bei Serialisierbarkeit nicht erlaubt wäre)
- Zertifizierungssperre – muss dann aber auf alle lesenden Transaktionen warten
- Nachteil: warten, Verklemmungen sind möglich (siehe vorherige Verfahren)
- Vorteil: keine Kaskadenabbrüche
- Aktuell unterstützt von mySQL als auch PostgreSQL und den bekannten Anbietern

```sql
-- Testtabelle erstellen
DROP TABLE IF EXISTS mvcc_demo;
CREATE TABLE mvcc_demo (
    id INT PRIMARY KEY,
    wert INT
);

-- Testdaten
INSERT INTO mvcc_demo VALUES (1, 100), (2, 200), (3, 300);

```

```sql
-- Sitzung 1:
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
START TRANSACTION;

-- Snapshot erstellen
SELECT * FROM mvcc_demo;  -- 100, 200, 300

```

```sql
-- Sitzung 2:
UPDATE mvcc_demo SET wert = 999 WHERE id = 2;
COMMIT;

```

```sql
-- Sitzung 1:
SELECT * FROM mvcc_demo;  -- 100, 200, 300
-- Keine Änderungen bevor Sitzung 1 nicht comitted


COMMIT;

-- Jetzt:
SELECT * FROM mvcc_demo;  -- 100, 999, 300

```

→ ermöglicht konsistentes Lesen ohne Sperren der Tabellenzeilen
(MySQL verwendet @@autocommit = 1 als Default)

Vielen Dank für Ihre Aufmerksamkeit!